Основы работы с модулями. Контроль загруженности процессора для предотвращения DoS

Знакомство с программированием и информационной безопасностью

В этом уроке

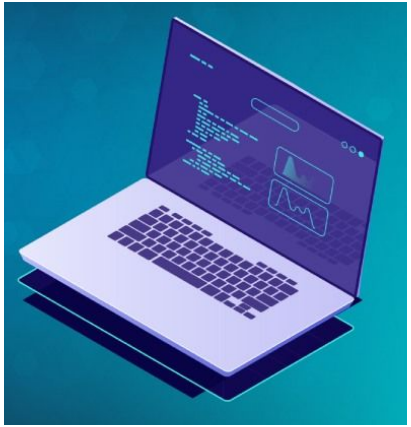
- Что такое модули?
- Как написать свой модуль?
- Какие есть встроенные модули?
- Как устанавливать модули?

Написание своего модуля

Как создать модуль?

Для создания собственного модуля в Python нужно сохранить ваш скрипт с расширением *.py.

После этого он станет доступным в любом другом файле.



Модуль __main__

Как скачать модуль?

```
if __name__ == '__main__':
    pass
```

Атрибуты модулей:

__name__ Полное имя модуля

__doc__ Строка документации

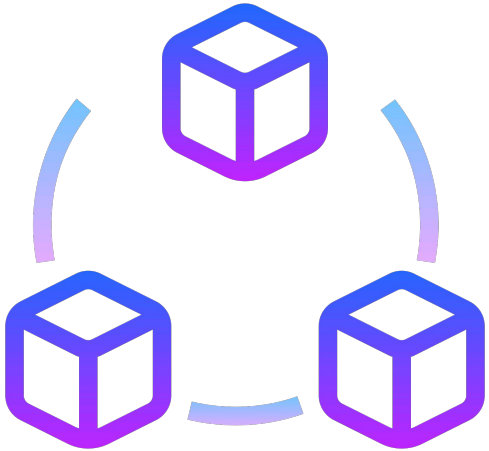
__dict__ Словарь модуля

__file__ Файл, в котором модуль определяется

__package__ Имя содержащегося пакета (если есть)

__path__ Список подкаталогов для поиска подмодулей пакета

__annotations__ Аннотации типов уровня модуля



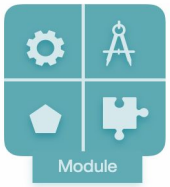
Импорт модулей

Модуль – файл python (*.py), в котором хранятся переменные, константы и функции.

```
import module_1[, module_2[,... module_N]
```

Как узнать все встроенные модули?

>>> print(help("modules"))



Python
Modules

Установка модулей

Как скачать модуль?

Команды установки модулей

>>> pip install module_name
>>> pip uninstall module_name

Список доступных модулей
https://pypi.org/

Чтобы работал pip из PyCharm, при его установке должна была стоять галочка «**Add to PATH**»

Нужен интернет

Основные модули

Какие есть модули?

print(help("modules"))

Модуль **random**

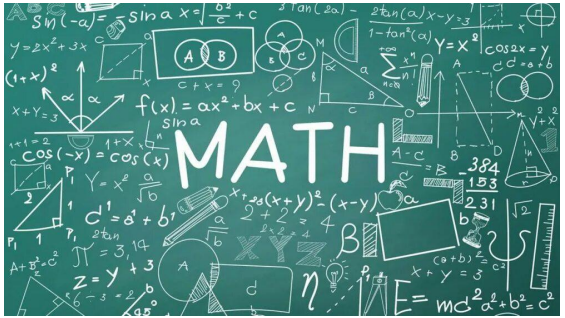
Модуль **math**

Модуль **datetime**

Модуль math

Модуль **math** оптимизирован для работы с числами с плавающей точкой (тип float) и предлагает более широкие возможности, чем встроенные операторы Python (операторы +, -, *, /, ** реализованы в языке, но не покрывают всех потребностей).

Степенные функции, корни
math.pow(x, y) — возвращает x**y как float (но может быть менее точным, чем x**y).
math.sqrt(x) — возвращает √x, x ≥ 0. Эквивалентно x**0.5, но точнее.
math.isqrt(n) — целый квадратный корень: наибольшее целое k, такое что $k^2 \le n$ (Python 3.8+)



Модуль math

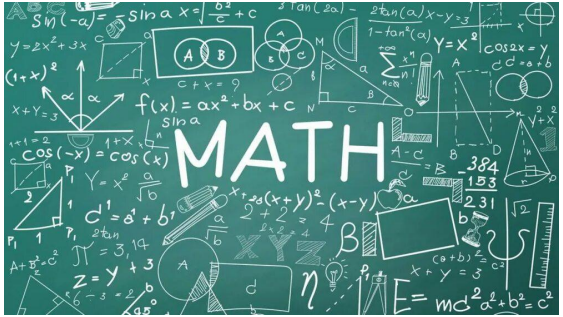
math.sin(x), math.cos(x), math.tan(x) — базовые функции.

Обратные: **math.asin(x), math.acos(x), math.atan(x)**.

math.exp(x) — e^x. Натуральный логарифм: **math.log(x).**

Логарифм по основанию 10: **math.log10(x).**

math.factorial(n) – точный факториал неотрицательного целого n



Модуль random

Модуль **random** в Python нужен, когда ты хочешь, чтобы программа делала что-то случайное или непредсказуемое

Примеры, где он может пригодиться:

Игры 🎲
Сделать кубик, который выдает случайное число (1-6). Выбрать случайного победителя в лотерее. Создать карту, которая случайно появляется на поле. Задать случайную погоду в игре.

Тесты и викторины 📝
Перемешать вопросы, чтобы они шли в разном порядке. Выбрать случайный вопрос из списка.

Генерация паролей 🔑
Создать сложный пароль из случайных букв и цифр.

Модуль random

random.random() – возвращает число из диапазона [0;1.0]

randint(a, b) – число из диапазона от a до b

choice(seq) – случайный элемент из последовательности (список, кортеж, строка).

sample(population, k) – выбор без возвращения: возвращает список длины k с уникальными элементами.

shuffle(list_) – перемешивает список in-place

Модуль datetime

date — календарная дата (год, месяц, день).

time — время суток (часы, минуты, секунды, микросекунды).

datetime — сочетание date + time.

timedelta — разница между двумя date, time или datetime, хранит дни, секунды, микросекунды.

tzinfo — базовый класс для часовых поясов, чаще используется его подкласс timezone

Модуль datetime

datetime.now() – текущее дата и время

date.today() – только дата (без времени)

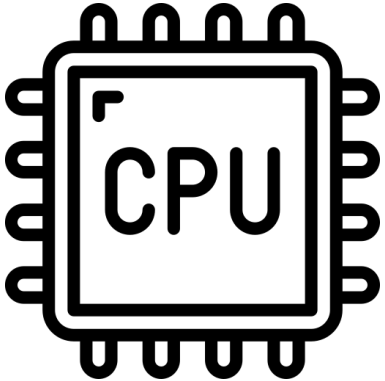
timedelta — разница между датами

timedelta — разница между датами

Модуль psutil

Библиотека для получения информации о системе:

- Загрузка CPU, памяти, диска, сети.
- Работа с процессами (запуск, завершение, анализ).
- Кроссплатформенность (Windows, Linux, macOS).



Модуль psutil

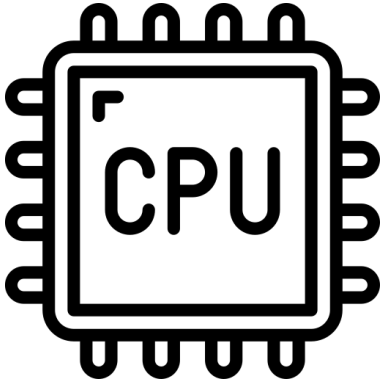
```python
import psutil

# Загрузка CPU (%)
cpu_usage = psutil.cpu_percent(interval=1)

# Использование оперативной памяти (в байтах)
mem = psutil.virtual_memory()

# Дисковая активность
disk = psutil.disk_usage('/')

# Список процессов
processes = psutil.process_iter()
```



Модуль os

Назначение модуля `os`:

- Взаимодействие с файловой системой (создание, удаление, переименование файлов и папок).
- Управление переменными окружения.
- Запуск системных команд и управление процессами.

Модуль os

```python
import os

# Работа с файлами и директориями
os.mkdir("folder")          # Создать папку
os.rename("old.txt", "new.txt")  # Переименовать файл
os.remove("file.txt")       # Удалить файл

# Получение информации
current_dir = os.getcwd()   # Текущая рабочая директория
files = os.listdir()        # Список файлов в директории

# Переменные окружения
path = os.getenv("PATH")    # Получить переменную окружения
```

Итоги урока

- Разобрались что такое модули
- Узнали как написать свой
- Изучили как использовать чужие модули на примере math, random, datetime